

BFP Rosario GIS: System at Hardware

Ipinapaliwanag ng dokumentong ito kung **paano gumagana** at **paano ginagamit** ang buong BFP Rosario GIS system. May tatlong bahagi ito:

1. Ang **app** (phone, computer, at website)
2. Ang **Firebase**, ang "backend" o online database
3. Ang **ESP32 incident alarm**, ang device sa fire station na nagpapatunog ng siren kapag may nag-report

Isinulat ito para sa mga estudyante, kaya ipinapaliwanag muna ang ideya bago ang technical na detalye.

1. Pangkalahatang ideya (System overview)

1.1 Ang problema na sinosolusyunan

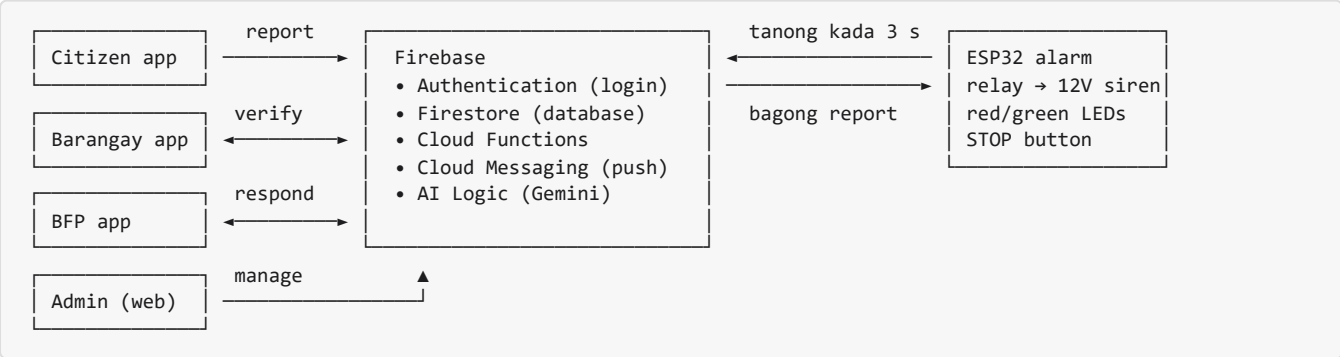
Kapag may sunog o emergency sa Rosario, Batangas, kailangang malaman agad ng BFP (Bureau of Fire Protection) kung **ano**, **gaano kalala**, at **saan eksakto**. Sa system na ito:

- Ang **citizen** ay nagre-report gamit ang app, kasama ang **GPS location** at **litrato**.
- Lumalabas agad ang report sa **mapa** ng barangay staff, BFP, at admin.
- Sabay nito, **tumutunog ang siren sa fire station**, kahit walang nakabukas na app.

1.2 Ang tatlong bahagi

Bahagi	Ginamit	Saan tumatakbo
BFP Rosario GIS app	Flutter (Android, iOS, web, Windows, macOS, Linux)	Phone ng citizens, device ng barangay at BFP, admin website
Backend	Firebase: Authentication, Firestore, Cloud Functions, Cloud Messaging, Hosting, AI Logic (Gemini)	Google Cloud, project <code>gis-cross-platform</code>
Incident alarm	ESP32 + relay + 12 V siren, naka-12 V UPS	BFP Rosario fire station

Isipin ang **Firebase** bilang **gitnang "bulletin board"**. Lahat ng bahagi ay nagbabasa at nagsusulat dito. Hindi direktang nag-uusap ang app at ang ESP32. Pareho silang dumadaan sa Firebase.



Ang mapa ay gawa sa `flutter_map` at OpenStreetMap, at ang mga lugar at address ay hinahanap gamit ang Photon at Nominatim. Nakasentro ang mapa sa Rosario, at tinatanggi ang report na nasa labas ng munisipyo.

2. Ang app at ang backend

2.1 Mga user at ang kanilang role

Bawat account ay may **role**, na naka-save sa Firestore sa `users/{uid}`, field `role`. Ang role ang nagsasabi kung ano ang puwedeng gawin ng account. Ang **Security Rules** ng Firestore ang nagbabantay nito.

Role	Value ng role	Ano ang ginagawa
Citizen / User	resident	Nagre-register gamit ang government ID. Kapag verified na, puwede nang mag-report na may location at litrato. Sariling reports lang ang nakikita
Barangay Staff	barangay	Tinitingnan ang mga <code>Pending</code> na report at ginagawang <code>Verified</code> . Kapag na-verify, naaabisuhan ang lahat ng BFP account
BFP Personnel	bfp	Tumatanggap ng alerts, at ina-update ang report habang rumeresponde (Dispatched → On Scene → Contained → Resolved → Closed)
System Admin (web)	admin	Nag-a-approve o nagre-reject ng citizen ID, nagro-route ng incident sa BFP, at gumagawa ng barangay, BFP at resident accounts
Alarm device	bfp	Ang sariling account ng ESP32. Nagbabasa lang ito ng <code>incidents</code>

2.2 Paano ginagamit: step by step

Citizen

1. Mag-register sa app at i-scan ang government ID.
2. Hintayin na i-approve ng admin ang account.
3. Para mag-report: buksan ang report form, piliin ang type, isulat ang detalye, at piliin ang urgency (Low, High, Critical).
4. I-set ang location (nasa ilalim mismo ng Location field ang mapa):
 - **GPS:** kunin ang kasalukuyang lokasyon ng phone.
 - **Search:** i-type ang lugar. Habang nagta-type, may lumalabas na mga mungkahi (type-ahead). Puwede rin ang Tagalog, gaya ng "palengke" o "munisipyo".
 - **Pin sa gitna (parang Grab):** nakapirmi ang pin sa gitna ng mapa. I-drag ang mapa hanggang tumapat ang pin sa eksaktong lugar. Puwede ring i-tap ang mapa.
5. Magdagdag ng litrato kung meron, tapos i-submit.

Barangay Staff

1. Buksan ang verification queue.
2. Tingnan ang `Pending` na report (detalye, litrato, lugar sa mapa).
3. I-verify kung totoo. Makatanggap agad ng push notification ang BFP.

BFP Personnel

1. Makakakita ng "New incident report" alert sa app, at tutunog ang siren sa station.
2. Pindutin ang **STOP** sa alarm box para patahimikin ang siren.
3. Sa app, i-update ang status habang rumeresponde: Dispatched, On Scene, Contained, Resolved, Closed. Sa Resolved at Closed, maglagay ng response notes.

Admin (sa website)

1. Sa **Verification** page, i-approve o i-reject ang mga bagong citizen (tingnan ang 2.3).

- 2. I-route ang incident sa BFP kung kailangan.
- 3. Gumawa ng accounts, kasama ang account ng **alarm device** (role: BFP Personnel).

2.3 Pag-verify ng citizen

- 1. Nagre-register ang citizen at ini-scan ang government ID (Philippine National ID, Driver License, Passport, UMID, o PhilHealth ID).
- 2. Tinitingnan ito ng AI (Gemini) kung malinaw at mukhang totoong ID. **Advisory lang ito.**
- 3. Ang **admin** ang nag-a-approve o nagre-reject. Ang verified citizen lang ang makakapagbukas ng report form.

Ginagawa ito para mabawasan ang **prank o pekeng report**.

Ano ang nakikita ng admin sa Verification queue:

- Sa bawat citizen: maliit na litrato (thumbnail) ng na-scan na ID, ang **pangalan, email**, barangay, at uri ng ID. Nasa **ilalim** ng mga detalye ang mga button, para buo ang lapad ng pangalan at email kahit sa phone.
- **View ID:** binubuksan nang malaki ang ID. Puwedeng i-pinch o i-scroll para mag-zoom. Kasama rito ang phone, birthdate, at ang resulta ng AI review ni Gemini, para maikumpara ng admin ang ID sa mga detalyeng isinulat ng citizen.
- **Approve / Reject:** nasa listahan mismo at nasa loob ng View ID. Hindi papayagan ang Approve kapag walang valid na ID image.

Ang ID image ay naka-save sa `users/{uid}`, field `governmentIdImage`, at ang AI review sa `governmentIdAiReview`.

2.4 Ano ang laman ng isang report

Kapag nag-submit ang citizen, gumagawa ang app ng bagong document sa `incidents` collection ng Firestore. Ito ang mga pangunahing field:

Field	Ibig sabihin
<code>type, description</code>	Uri ng insidente at detalye
<code>priority</code>	<code>Low, High</code> o <code>Critical</code> (tinatanggap din ng rules ang <code>Medium</code> para sa lumang data)
<code>latitude, longitude</code>	Lokasyon: galing GPS, tap sa mapa, o address search
<code>barangayName, barangayId</code>	Barangay na natukoy mula sa lokasyon
<code>evidenceImage</code>	Litrato (optional), naka-save sa loob ng document
<code>aiAssessment</code>	Resulta ng pag-assess ng Gemini sa litrato
<code>uid, reporterId, reporterName, phone, address</code>	Impormasyon ng nag-report
<code>status</code>	Laging <code>Pending</code> kapag bagong gawa
<code>createdAt, updatedAt</code>	Oras galing sa server. <code>createdAt</code> ang binabantayan ng ESP32

Tungkol sa priority at AI: kapag may litrato, tinatantya ng Gemini ang severity nito. Ang severity na iyon ang **pinakamababang priority** na puwedeng piliin ng citizen. Halimbawa, kapag sinabi ng AI na Critical, hindi puwedeng i-submit bilang Low. Pinapatupad din ito ng Security Rules.

2.5 Ang buhay ng isang report (status)



Naka-record kung sino ang nag-update sa bawat hakbang.

2.6 Paano naabisuhan ang mga tao

Alert	Paano gumagana	Limitasyon
Alert dialog sa app	Sa BFP at admin screens, binabantayan ng app ang mga <code>Pending</code> report at tumutunog kada 2 segundo hanggang i-acknowledge	Gumagana lang habang bukas ang app
Push notification	Kapag na-verify ng barangay ang report, may <code>notifications</code> document na nagagawa para sa bawat BFP user. Ang Cloud Function na <code>sendBfpEmergencyPush</code> ang nagpapadala ng push (FCM)	Kailangang naka-install ang app at naka-on ang notifications
Siren ng ESP32	Tinatanong ng alarm box ang Firestore kada 3 segundo kung may bagong report, at pinapatunog ang siren	Kailangan ng Wi-Fi at kuryente (sagot ng UPS ang brownout)

Ang ESP32 ang **"laging gising"** na alert ng station. Gumagana ito kahit walang nakabukas na phone o computer.

Personnel on duty. Sa Admin at BFP dashboards, may live na bilang ng mga BFP account na **online** ngayon. Isipin ito na parang "attendance" na kusang nag-a-update:

- Habang bukas ang app ng isang BFP personnel, nagsusulat ito ng "heartbeat" kada **60 segundo** sa `presence/{uid}`.
- Binibilang na online ang account kapag may heartbeat ito sa nakaraang **150 segundo**. Kapag isinara ang app, kusang nawawala ito sa bilang pagkalipas ng ilang minuto.
- Ayon sa Security Rules, sariling `presence` document lang ang puwedeng isulat ng bawat BFP account.

2.7 Paano kumokonekta ang ESP32 sa Firebase

- May **sariling email at password account** ang ESP32, na ginawa ng Super Admin sa **Accounts** page na may role na **BFP Personnel**. Kailangan ito kasi BFP, barangay at admin lang ang pinapayagan ng rules na magbasa ng lahat ng incident.
- Gumagamit ito ng **Firebase REST API** (parang pag-access sa website gamit ang HTTPS). Walang Firebase app na naka-install sa ESP32.
- Nagbabasa lang** ito ng `incidents`. Hindi ito nagsusulat sa Firestore.
- Tumutunog agad ito sa bawat bagong report**, kahit `Pending` pa at hindi pa verified ng barangay. Ginagawa ito para makapaghandag agad ang station.
- Ang email at password ng device ay nasa `hardware/esp32_incident_alarm/secrets.h`. Hindi ito naka-upload sa git, para hindi makita ng iba.

3. Ang ESP32 incident alarm (hardware)

Ang incident alarm ay isang maliit na box sa fire station. Laging naka-on ito (may 12 V UPS para sa brownout), binabantayan ang Firebase, at pinapatunog ang 12 V siren kapag may bagong report.

3.1 Mga piyesa (components)

Piyesa	Trabaho
ESP32 Dev Module (ESP32-D0WD-V3)	Ang "utak": Wi-Fi, pagtanong sa Firebase, LEDs, button, at kontrol ng relay
SRD-05VDC-SL-C 1-channel relay module	Parang switch na kinokontrol ng ESP32 . Ito ang nagbubukas at nagsasara ng kuryente papunta sa siren
LTE-1101J siren, 12 V DC	Ang malakas na tunog ng alarm
12 V DC mini UPS	Pangunahing power. Tuloy ang alarm kahit brownout
Buck converter (LM2596-type), naka-set sa 5.0 V	Pinapababa ang 12 V ng UPS papuntang 5 V para sa ESP32
Red LED + 220 ohm resistor	Ilaw ng alarm, at ng Wi-Fi setup (kumukurap)
Green LED + 220 ohm resistor	Ilaw ng "ready" o online
4-pin push button	STOP button: pinapatahimik ang alarm, at binubuksan o sinasara ang Wi-Fi setup kapag hinawakan nang matagal
Breadboard at jumper wires	Mga koneksyon

Optional ang HW-131 / MB102 breadboard power module. Puwede itong pamalit sa buck converter kapag nagtest, pero umiinit ito sa 12 V, kaya buck converter ang ginamit sa final setup.

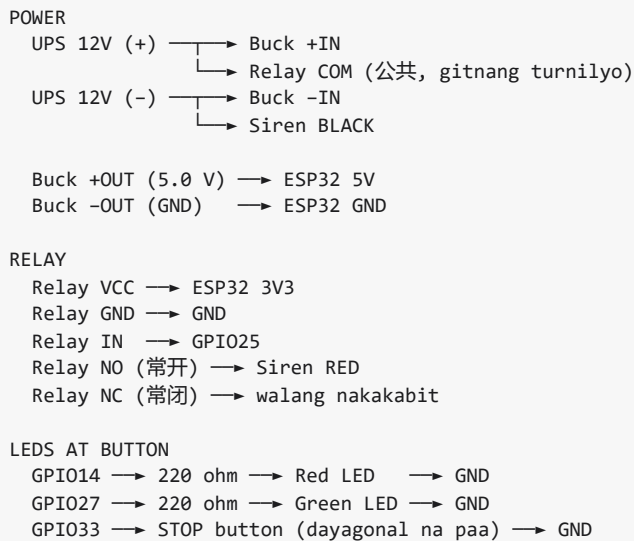
Bakit may relay? Ang GPIO pin ng ESP32 ay 3.3 V lang at kaunting kuryente ang kaya. Ang siren ay 12 V at mas malakas humigop ng kuryente. Kaya ang ESP32 ay "pumipindot" lang sa relay, at ang relay ang nagkakabit ng 12 V sa siren.

3.2 Pin assignment

ESP32 pin	Nakakabit sa	Paalala
GPIO25	Relay IN	Direktang wire. LOW = relay ON (siren), HIGH = relay OFF
GPIO14	Red LED (+) sa 220 ohm	Ang LED (–) ay sa GND
GPIO27	Green LED (+) sa 220 ohm	Ang LED (–) ay sa GND
GPIO33	STOP button, isang paa	Ang dayagonal na paa ay sa GND. Internal pull-up ang gamit kaya walang resistor
5V	Buck converter +OUT (5.0 V)	Power ng ESP32 kapag galing UPS
3V3	Relay VCC	Tingnan ang 3.4 kung bakit sa 3V3 ang relay
GND	Buck –OUT, relay GND, LED (–), button	Iisa ang GND ng lahat

Naka-reserve ang GPIO26 sa firmware para sa optional na buzzer. Walang buzzer sa final build, kaya siren lang ang tunog.

3.3 Wiring



Chinese ang nakasulat sa screw terminals ng relay: 常开 = **NO** (normally open), 公共 = **COM** (common), 常闭 = **NC** (normally closed). Nasa **NO** ang siren, kaya tahimik ito hangga't hindi naka-ON ang relay.

3.4 Mga natutunan habang nagte-test

- **I-set muna sa 5.0 V ang buck converter bago ikabit ang ESP32.** Madalas mas mataas sa 5 V ang default nito at puwedeng masunog ang ESP32. May button ang display para lumipat sa IN o OUT reading.
- **Relay ang nagpapagana sa siren, hindi ang GPIO.** Masyadong malakas humigop ng kuryente ang siren at 12 V ito.
- **Sa 3V3 ang VCC ng relay, hindi 5 V.** Walang H/L trigger jumper ang relay na ito. Noong nasa 5 V ang VCC, kaya ng 3.3 V signal ng ESP32 na i-ON ang relay pero hindi ito ma-OFF nang tuluyan, kaya hindi tumitigil ang siren. Sa 3V3, maayos na nag-o-ON at OFF ang relay (LOW = ON, HIGH = OFF).
- **Pinahina ang Wi-Fi transmit power.** Iisang 3.3 V supply ang gamit ng relay coil at ng Wi-Fi ng ESP32. Sa buong lakas ng Wi-Fi, umiilaw ang LED ng relay pero hindi gumagalaw ang coil. Ibinaba ng firmware ang Wi-Fi power sa 8.5 dBm, at gumagana na ang siren kahit online. Kung malayo ang router at nawawala ang koneksyon, kakailanganin ng transistor o ng 3.3 V-trigger relay/MOSFET module.
- **Dayagonal na paa ng 4-pin button ang gamitin.** Laging magkakonekta ang dalawang paa sa iisang gilid, kaya iisipin ng ESP32 na laging pinipindot ang button, at bubukas ang Wi-Fi setup sa bawat boot.
- **Iisang GND.** Magkakonekta ang GND ng ESP32, relay at buck converter.
- **Huwag pagsabayin ang USB at ang buck sa ESP32.** Kapag mag-a-upload ng code gamit ang USB, tanggalin muna ang wire ng buck +OUT → ESP32 5V.

3.5 Mga ilaw at kontrol

Nakikita mo	Ibig sabihin
Steady green	Ready: online at nagbabantay ng bagong report
Mabagal na kurap ng green	Kumokonekta sa Wi-Fi/Firebase, o may error
Kumukurap ang red, patay ang green	Bukas ang Wi-Fi setup hotspot
Red + siren	May bagong incident report. Pindutin ang STOP para tumahimik

Gawin	Mangyayari
Pindutin ang STOP	Titigil ang alarm hanggang sa susunod na bagong report
Hawakan ang STOP nang 5 segundo	Bubukas ang Wi-Fi setup hotspot na BFP-Alarm-Setup (kumukurap ang red)
Hawakan ulit nang 5 segundo	Sasara ang hotspot, papatayin ang radio nito, at babalik sa green
Hawakan ang STOP habang nag-o-on	Bubukas agad ang Wi-Fi setup

Kusa ring nagsasara ang setup hotspot pagkalipas ng 3 minuto kung walang na-save.

3.6 Paano palitan ang Wi-Fi

1. Hawakan ang STOP nang 5 segundo. Kukurap ang red LED.
2. Sa phone, kumonekta sa BFP-Alarm-Setup (ang password ay `SETUP_AP_PASSWORD` sa `secrets.h`).
3. Kusang bubukas ang setup page. Kung hindi, buksan ang `http://192.168.4.1` sa browser.
4. Pindutin ang **Configure WiFi**, piliin ang network (2.4 GHz lang), ilagay ang password, at i-Save.
5. Kokonekta ang ESP32 sa bagong network, mamamatay ang hotspot, at magiging steady green.

4. Ang firmware (code ng ESP32)

File	Para saan
<code>hardware/esp32_incident_alarm/esp32_incident_alarm.ino</code>	Ang totoong alarm firmware
<code>hardware/esp32_incident_alarm/secrets.h</code>	Password ng setup hotspot, Firebase API key at project ID, email at password ng device account. Hindi naka-upload sa git; kopyahin mula sa <code>secrets.example.h</code>
<code>hardware/esp32_wiring_test/esp32_wiring_test.ino</code>	Pang-test ng wiring nang walang Wi-Fi: green pagka-on, pindot = red + siren, pindot ulit = green

Libraries: **ArduinoJson 7** at **WiFiManager (tzapu)**. Board: **ESP32 Dev Module**.

4.1 Paano gumagana ang firmware

May **dalawang core** (dalawang "processor") ang ESP32. Hinati ang trabaho sa dalawa para hindi ma-freeze ang siren o ang STOP button kapag mabagal ang internet.

Core 0: network task (kada 3 segundo)

1. Kumonekta sa naka-save na Wi-Fi gamit ang WiFiManager, o buksan ang setup hotspot.
2. Mag-login sa Firebase Authentication gamit ang device account (`accounts:signInWithPassword`). Nire-renew ang ID token kada 50 minuto.
3. Sa unang matagumpay na query, tandaan ang `createdAt` ng pinakabagong report bilang simula. Kaya hindi tumutunog ang mga lumang report tuwing nag-o-on ang device.
4. Magtanong sa Firestore sa `incidents`: "may report ba na `createdAt > lastSeen`?", naka-order ayon sa `createdAt`, at `priority`, `type`, `barangayName` at `createdAt` lang ang kinukuha.
5. Sa bawat bagong report, i-update ang `lastSeen` at mag-request ng alarm ayon sa severity.

Core 1: alarm loop (kada 10 milliseconds)

- Simulan ang alarm (relay ON, red LED) kapag may bagong report. Kapag may mas seryosong report, nag-a-upgrade ang alarm; hindi ito bumababa.
- Basahin ang STOP button (50 ms debounce; 5 segundong hawak para sa Wi-Fi setup).

- Paandarin ang LEDs ayon sa table sa 3.5.
- Tanggapin ang Serial test commands.

Bakit "polling" kada 3 segundo? Mas simple ito para sa ESP32 kaysa sa real-time listener, at maliit ang data na kinukuha kada tanong. Ang pinakamatagal na delay mula report hanggang siren ay mga 3 segundo, dagdag pa ang oras ng internet.

4.2 Severity

Ang **priority** ng report ang nagdedesisyon ng pattern ng alarm:

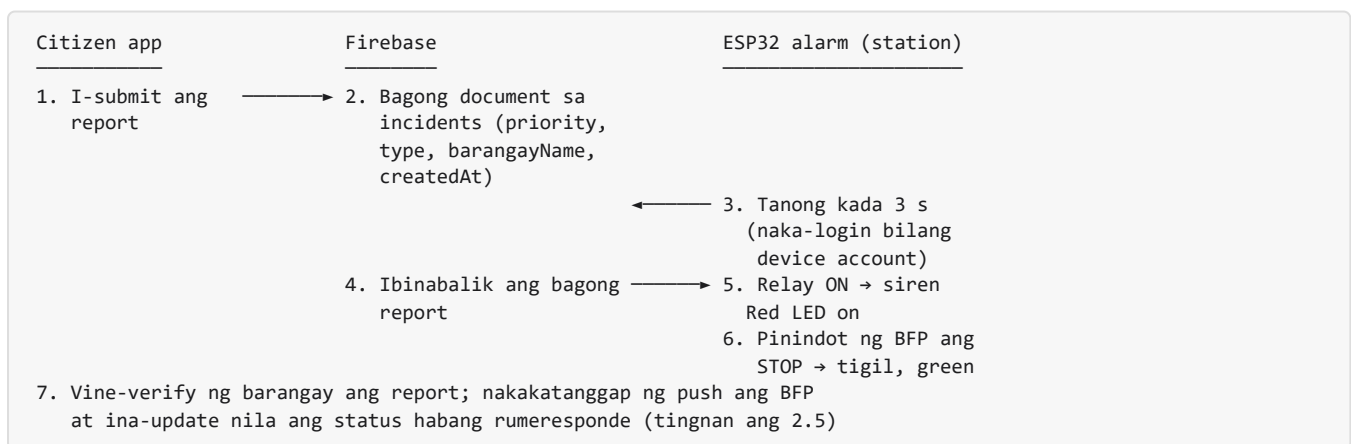
priority	Severity	Red LED
Critical	Critical	Steady
High (o hindi kilala)	High	Mabilis na kurap
Medium / Low	Low	Mabagal na kurap

Pareho ang tunog ng siren sa lahat ng severity, kasi iisang tono lang ang LTE-1101J. Magagamit din ang mga pattern na ito sa buzzer sa GPIO26 kung magdadagdag nito.

4.3 Pag-upload at pag-test

- **Arduino IDE:** buksan ang sketch, piliin ang board na **ESP32 Dev Module** at port na **COM7**, tapos i-Upload.
- **Terminal:** `arduino-cli compile --upload -p COM7 --fqbn esp32:esp32:esp32 hardware/esp32_incident_alarm` (kasama na ang `arduino-cli` sa Arduino IDE).
- **Serial Monitor** (115200 baud, line ending **New Line**): i-type ang **1**, **2** o **3** at Enter para sa Low, High o Critical na test alarm, at **0** para patigilin. Isang digit lang bawat linya, para hindi makapagpatunog ng siren ang electrical noise sa serial pin.
- Ipinapakita ng boot log ang dahilan ng pag-restart (halimbawa **BROWNOUT** kapag bumaba ang power).

5. Buong daloy: mula report hanggang siren



Ang pinakamatagal na delay mula sa pag-submit hanggang tumunog ang siren ay mga **3 segundo**, dagdag pa ang oras ng internet.

6. Troubleshooting

Problema	Posibleng dahilan	Ayos
Laging mabagal na kurap ang green	Walang Wi-Fi o hindi maka-login sa Firebase	Tingnan ang Serial log. Hawakan ang STOP nang 5 segundo para pumili ng ibang network. 2.4 GHz lang
Kumukurap ang red sa bawat boot (kusang bumubukas ang setup)	Laging "pinipindot" ang STOP button	Gamitin ang dayagonal na paa ng button
Hindi tumitigil ang siren	Nasa 5 V ang relay VCC, o nasa NC ang siren	Relay VCC sa 3V3; siren sa NO
Umiilaw ang relay pero walang siren	Kulang ang 3.3 V para sa coil, o maluwag ang COM/NO wire	Higpitan ang turnilyo; panatilihin mahina ang Wi-Fi power, o gumamit ng transistor / 3.3 V-trigger module
Walang ilaw ang display ng buck	Naka-OFF ang UPS, baligtad ang polarity, o maluwag ang wire	I-ON ang UPS, i-check ang + at -, at tanso ang ipitin (hindi goma)
permission denied sa log	Walang BFP Personnel role ang device account	I-set ang role sa admin panel
Walang COM7 kapag mag-a-upload	Tanggal ang USB o pang-charge lang ang cable	Gumamit ng data cable; isara ang Arduino Serial Monitor

7. Technologies used (Mga ginamit na technology)

Listahan ng lahat ng ginamit, at paliwanag ng bawat isa

Ito ang lahat ng technology na ginamit sa system. Sa bawat isa, may sagot sa tatlong tanong: **Ano ito?**, **Para saan sa system?**, at **Paano ito gumagana rito?**

A. AI at mapa

1. Google Gemini AI (sa pamamagitan ng Firebase AI Logic)

- **Ano ito:** Isang AI model ng Google na kayang "tumingin" at umintindi ng litrato at text. Model na `gemini-3.5-flash-lite` ang gamit, dahil mabilis ito (mga 2 segundo) at may libreng tier.
- **Para saan:** (a) Sa registration, tinitingnan nito kung malinaw at mukhang totoong government ID ang na-scan. (b) Sa report, tinatantya nito kung gaano kalala ang litrato ng insidente (Low, High o Critical) at kung anong uri ito.
- **Paano:** Ipinapadala ng app ang litrato kay Gemini gamit ang `firebase_ai` package, at sumasagot ito ng resulta. Ang severity na sagot nito ang pinakamababang priority na puwedeng piliin ng citizen. Halimbawa, kapag Critical ang sabi ng AI, hindi puwedeng i-submit bilang Low.
- **Tandaan:** Advisory lang ang AI, parang tagapayo. Tao pa rin (ang admin) ang nagdedesisyon kung aaprubahan ang citizen.

2. Nominatim

- **Ano ito:** Libreng serbisyo ng OpenStreetMap para sa **geocoding**, o ang pagpapalit ng coordinates sa address at ng address sa coordinates.
- **Para saan:** Kapag GPS ang ginamit o kapag naglagay ng pin sa mapa, Nominatim ang nagsasabi kung anong address at barangay iyon. Backup din ito ng paghahanap ng lugar kapag walang resulta ang Photon.

- **Paano:** Nagpapadala ang app ng request sa `nominatim.openstreetmap.org` na may latitude at longitude, at sumasagot ito ng address. Parang nagtanong ka sa isang kartero: "Anong address ng bahay na ito?"

3. OpenStreetMap (OSM)

- **Ano ito:** Isang libreng mapa ng buong mundo na ginawa ng mga volunteer, parang "Wikipedia ng mapa".
- **Para saan:** Dito galing ang itsura ng mapa sa app: mga kalsada, gusali at pangalan ng lugar. Galing din sa OSM ang data na hinahanap ng Nominatim at Photon.
- **Paano:** Hinahati ang mapa sa maliliit na litrato na tinatawag na **tiles**. Kinukuha ng app ang tiles na kailangan habang ini-scroll o zino-zoom ang mapa.
- **Bakit hindi Google Maps:** Libre ang OSM at hindi kailangan ng bayad na API key.

4. Photon

- **Ano ito:** Isang search engine para sa mga lugar sa OpenStreetMap, gawa ng komoot. Ginawa ito para sa **type-ahead**, ibig sabihin may lumalabas nang mungkahi habang nagta-type ka pa lang.
- **Para saan:** Sa report form, i-type lang ng citizen ang lugar, gaya ng "Jollibee" o "mcdo", at may lalabas na listahan ng mga posibleng lugar sa Rosario.
- **Paano:** Nagpapadala ang app ng tynype sa `photon.komoot.io`, naka-limit sa paligid ng Rosario. May dagdag na **Tagalog aliases** ang app: ginagawang "public market" ang "palengke" at "municipal hall" ang "munisipyo" bago maghanap. Nasa `lib/core/services/place_search_service.dart` ang code.

5. flutter_map

- **Ano ito:** Isang Flutter package na nagpapakita ng mapa sa loob ng app.
- **Para saan:** Ito ang "screen" ng mapa. Dito ipinapakita ang OpenStreetMap tiles, ang mga marker ng incident, ang fire station, at ang coverage circle nito.
- **Paano:** May kulay ang bawat marker ayon sa priority at status: pula ang Critical, amber ang High, asul ang Verified, berde ang Resolved/Closed. Sa report form, nakapirmi ang pin sa gitna (parang Grab), at ang mapa ang dina-drag ng citizen.

B. Ang app

6. Flutter (Dart)

- **Ano ito:** Framework ng Google para gumawa ng app. Dart ang programming language nito.
- **Para saan:** Isang code lang ang isinulat, pero tumatakbo ito sa Android, iOS, web, Windows, macOS at Linux. Ito ang dahilan kung bakit **cross-platform** ang app.
- **Paano:** Gawa sa maliliit na "widget" ang bawat screen, parang Lego na pinagdudugtong-dugtong. Nasa `lib/` ang code: `app`, `core` at `features`.

7. Firebase Authentication

- **Ano ito:** Serbisyo ng Firebase para sa accounts at login.
- **Para saan:** Dito naka-register ang lahat ng account: citizen, barangay, BFP, admin, at pati ang ESP32 alarm.
- **Paano:** Kapag nag-login, binibigyan ng Firebase ang user ng **token**, parang ID pass. Ipinapakita ang token na iyon tuwing magbabasa o magsusulat sa database, para malaman kung sino siya.

8. Google Sign-In

- **Ano ito:** Pag-login gamit ang Gmail account.
- **Para saan:** Para hindi na kailangang gumawa ng bagong password ang citizen.
- **Paano:** Sa phone, `google_sign_in` package ang gamit. Sa website, may lalabas na popup ng Google. Pagkatapos, ipinapasa ang resulta sa Firebase Authentication.

9. Facebook Login

- **Ano ito:** Pag-login gamit ang Facebook account.
- **Para saan:** Isa pang madaling paraan ng pag-login, dahil marami ang may Facebook.
- **Paano:** Sa phone, `flutter_facebook_auth` package ang gamit. Sa website, popup ang lalabas. Ipinapasa rin ang resulta sa Firebase Authentication.

10. Cloud Firestore

- **Ano ito:** Ang online database ng Firebase. Isa itong **NoSQL** database, at hindi relational (SQL). Nakaayos ang data sa **collections** (parang folder) at **documents** (parang file sa loob ng folder). Ang bawat document ay parang form na may mga field, kaya walang fixed na table at column na gaya sa MySQL.
- **Para saan:** Ito ang gitnang "bulletin board" ng system. Mga pangunahing collection:
 - `users` : profile at role ng bawat account, kasama ang litrato ng ID
 - `incidents` : lahat ng report
 - `notifications` : mga alert para sa BFP
 - `activity_logs` : record kung sino ang gumawa ng ano
 - `presence` : kung sinong BFP personnel ang online
- **Paano:** **Real-time** ito. Kapag may nagbago, kusang nag-a-update ang screen ng lahat ng nakatingin, nang hindi na kailangang i-refresh.

11. Firestore Security Rules

- **Ano ito:** Mga patakaran na nakasulat sa `firestore.rules`, na nagbabantay sa database.
- **Para saan:** Parang guwardiya sa pinto. Halimbawa: sariling report lang ang nakikita ng citizen, BFP at barangay lang ang nakakabasa ng lahat ng incident, at tinatanggihan ang report na nasa labas ng Rosario o sobrang laki ng litrato.
- **Paano:** Sinusuri ng Firebase ang bawat basa at sulat bago ito payagan, kahit pa may nagtangkang dumaan sa labas ng app.

12. Cloud Functions (Node.js 20)

- **Ano ito:** Code na tumatakbo sa server ng Google, hindi sa phone ng user.
- **Para saan:** Pagpapadala ng emergency push notification sa BFP (`sendBfpEmergencyPush`), at backup na ID review (`reviewCitizenIdWithAi`).
- **Paano:** Kusang tumatakbo ang function kapag may bagong document sa `notifications`. Parang alarm clock na tumutunog kapag may nangyari.

13. Firebase Cloud Messaging (FCM)

- **Ano ito:** Serbisyo ng Google para sa **push notification**, ang mga abiso na lumalabas sa phone kahit sarado ang app.
- **Para saan:** Inaabisuhan ang BFP personnel kapag na-verify ng barangay ang isang report.
- **Paano:** May sariling "address" (token) ang bawat phone. Doon ipinapadala ng Cloud Function ang abiso.

14. Firebase Hosting

- **Ano ito:** Lugar kung saan naka-host ang website.
- **Para saan:** Dito naka-upload ang web version ng app, sa `https://gis-cross-platform.web.app`. Dito pumapasok ang admin at ang mga gumagamit ng computer.
- **Paano:** Gagawin muna ang website gamit ang `flutter build web`, tapos ia-upload gamit ang Firebase CLI.

15. Google Drive

- **Ano ito:** Online storage ng Google.

- **Para saan:** Dito naka-upload ang Android APK. Ang "Download Android APK" button sa login page ay nakaturo sa file na ito.
- **Bakit:** Hindi tumatanggap ang Firebase Hosting ng `.apk` sa libreng Spark plan. Kapag may bagong APK, pinapalitan lang ang file gamit ang "Manage versions", kaya hindi nagbabago ang link.

C. Mga feature sa phone

16. Geolocator (GPS)

- **Ano ito:** Flutter package na kumukuha ng lokasyon mula sa GPS ng phone o sa browser.
- **Para saan:** Para makuha agad ang eksaktong lokasyon ng citizen kapag nagre-report.
- **Paano:** Hihingi muna ang app ng permiso sa location. Kapag pumayag, makukuha ang latitude at longitude, at ilalagay ito sa mapa.

17. Camera at ID auto-scanner

- **Ano ito:** Ang `camera` at `image` packages, at ang sariling scanner ng app na nasa `lib/features/auth/id_auto_scanner.dart`.
- **Para saan:** Pagkuha ng litrato ng ID sa registration, at ng litrato ng insidente sa report.
- **Paano:** Parang sa GCash. Tinitingnan ng app ang camera mga 4 na beses kada segundo. Kapag tumapat ang ID sa guide, sapat ang liwanag, walang silaw, malinaw ang text, at hindi gumagalaw ang phone nang 3 sunod-sunod na frame, kusa nitong kinukuhanan at kina-crop ang ID. Sa website, sa `<video>` ng browser kinukuha ang frames gamit ang `web` package.

18. Decision tree (rule-based na risk score)

- **Ano ito:** Isang serye ng "kung ganito, ganito" na tanong na nakasulat sa code (`lib/core/models/incident_risk.dart`). **Hindi ito AI.**
- **Para saan:** Nagbibigay ng risk score sa bawat incident, para makita ng staff kung alin ang mas dapat unahin.
- **Paano:** Nagbibigay ito ng puntos:
 - Ayon sa priority: Critical = 6, High = 3, Medium = 2, Low = 1
 - Structure o chemical fire: +2. Vehicle o electrical: +1
 - Kapag 5 o higit pang report sa parehong barangay kamakailan: +2, at +1 pa kapag 10 o higit
 - Kabuuan: 6 pataas = **High** risk, 4–5 = **Medium**, mas mababa = **Low**

Hindi nito binabago ang `priority` na pinili ng citizen. Dagdag na impormasyon lang ito para sa staff.

D. Hardware (ang alarm sa fire station)

19. ESP32

- **Ano ito:** Maliit at murang computer (microcontroller) na may Wi-Fi.
- **Para saan:** Ito ang "utak" ng alarm box. Binabantayan nito ang Firebase at pinapatunog ang siren kapag may bagong report.
- **Paano:** Kada 3 segundo, tinatanong nito ang Firestore kung may bagong report. Kapag meron, binubuksan nito ang relay para tumunog ang siren, at sinisindihan ang pulang ilaw.

20. Arduino (C++)

- **Ano ito:** Programming language at tools para sa mga microcontroller gaya ng ESP32.

- **Para saan:** Dito isinulat ang firmware ng alarm (`hardware/esp32_incident_alarm/esp32_incident_alarm.ino`).
- **Paano:** Kino-compile ang code at ina-upload sa ESP32 gamit ang Arduino IDE o `arduino-cli`, sa USB (COM7).

21. WiFiManager

- **Ano ito:** Arduino library para sa pag-set ng Wi-Fi nang hindi binabago ang code.
- **Para saan:** Kapag lumipat ng network, hindi na kailangang i-upload ulit ang firmware.
- **Paano:** Hawakan ang STOP nang 5 segundo. Gagawa ang ESP32 ng sariling hotspot na `BFP-Alarm-Setup`. Kumonekta rito gamit ang phone, buksan ang `http://192.168.4.1`, at piliin ang bagong Wi-Fi.

22. ArduinoJson

- **Ano ito:** Arduino library para sa **JSON**, ang format ng data na ginagamit ng Firebase.
- **Para saan:** Para mabasa ng ESP32 ang sagot ng Firebase (halimbawa ang `createdAt` at `priority` ng report), at para makagawa ng request.
- **Paano:** Ginagawa nitong mga variable na kayang basahin ng code ang text na galing Firebase.

23. Firebase REST API

- **Ano ito:** Paraan para makausap ang Firebase gamit lang ang HTTPS, parang pagbukas ng website.
- **Para saan:** Walang Firebase app na kayang tumakbo sa ESP32, kaya REST API ang gamit nito.
- **Paano:** Una, nagla-login ito gamit ang sariling device account (Identity Toolkit). Pagkatapos, nagtatanong ito sa Firestore gamit ang `runQuery`. **Nagbabasa lang** ito at hindi nagsusulat.

Nasa mga table sa ibaba ang buod ng bawat technology ayon sa grupo.

7.1 App framework

Technology	Para saan
Flutter (Dart)	Isang code lang para sa Android, iOS, web, Windows, macOS at Linux. Ito ang dahilan kung bakit "cross-platform" ang app
<code>provider</code>	State management: pag-manage ng data sa pagitan ng mga screen
<code>camera</code> , <code>image_picker</code> , <code>image</code>	Pagkuha at pag-compress ng litrato ng ID at ng incident. Ang <code>camera</code> at <code>image</code> din ang gamit ng ID auto-scanner (tingnan sa ibaba)
<code>web</code>	Para sa web version: pagbasa ng frames mula sa camera ng browser (<code>lib/core/web/video_frame_grabber*.dart</code>), at pag-alam kung Android o iPhone ang gamit (<code>lib/core/web/browser_device*.dart</code>) para maipakita ang tamang paraan ng pag-install
<code>google_sign_in</code> , <code>flutter_facebook_auth</code>	Pag-login gamit ang Google at Facebook account
<code>http</code>	Pag-connect sa mga online API, gaya ng Photon at Nominatim
<code>flutter_spinkit</code>	Mga loading animation
<code>flutter_launcher_icons</code> , <code>flutter_native_splash</code>	Icon ng app at splash screen kapag binubuksan ang app
<code>intl</code>	Format ng petsa at oras
<code>url_launcher</code>	Pagbukas ng tawag o link, halimbawa ang hotline ng fire station

ID auto-scanner (parang sa GCash). Sa registration, hindi na kailangang pindutin ang shutter. Tinitingnan ng app ang camera preview nang paulit-ulit, mga 4 na beses kada segundo:

- 1. Kailangang tumapat ang apat na gilid ng ID sa guide na nasa screen.
- 2. Dapat sapat ang liwanag, walang sobrang silaw (glare), malinaw ang text, at hindi gumagalaw ang phone.
- 3. Kapag tatlong sunod-sunod na frame ang pumasa, kusa nang kinukuhanan at kina-crop ang ID.
- 4. Mabilis na tinitingnan ni Gemini kung mukha ngang ID, para matanggihan agad ang hindi ID.

Kapag 30 segundo nang walang nakuha, pinapatay ang camera hanggang pindutin ang "Scan ulit". Nasa `lib/features/auth/id_auto_scanner.dart` ang code.

7.2 GIS at mapa

Technology	Para saan
<code>flutter_map</code>	Ang mismong mapa sa app at website. Libre at open source (hindi Google Maps)
OpenStreetMap tiles	Ang itsura ng mapa: mga kalsada at lugar. Libreng map data galing sa OpenStreetMap
<code>latlong2</code>	Coordinates (latitude at longitude) ng mga pin sa mapa

May kulay ang mga marker ng incident depende sa priority at status: pula ang Critical, amber ang High, asul ang Verified, at berde ang Resolved/Closed. Makikita rin sa mapa ang fire station at ang coverage circle nito.

7.3 GPS at location

Technology	Para saan
<code>geolocator</code>	Kinukuha ang GPS location ng phone o browser ng citizen kapag nagre-report
Photon (komoot, OpenStreetMap)	Type-ahead na place search sa report form (<code>lib/core/services/place_search_service.dart</code>). Naka-limit sa paligid ng Rosario, at kaya nito ang palayaw at typo (hal. "mcdo"). May Tagalog aliases din, gaya ng palengke → public market at munisipyo → municipal hall
Nominatim (OpenStreetMap)	Geocoding: ginagawang address ang coordinates (reverse). Backup din ito ng search kapag walang resulta ang Photon. Dito rin nalalaman kung anong barangay ang report
<code>geocoding</code>	Backup na address lookup sa phone

Sa mapa ng report form, nakapirmi ang pin sa gitna (parang Grab), at ang mapa ang dina-drag. Puwede ring i-tap ang mapa. Tinatanggihan ng security rules ang report na nasa labas ng Rosario.

7.4 AI (Artificial Intelligence)

Technology	Para saan
Google Gemini (<code>gemini-3.5-flash-lite</code>) gamit ang Firebase AI Logic (<code>firebase_ai</code>)	1. Pag-check ng government ID ng citizen sa registration: kung malinaw at mukhang totooang government ID. 2. Pag-assess ng litrato ng incident: tinatantya ang severity (Low / High / Critical) at ang type ng insidente. Ang severity galing sa AI ang pinakamababang priority na puwedeng piliin ng citizen
Gemini API (<code>gemini-2.0-flash</code>) sa Cloud Function	Backup na ID review sa server side (<code>reviewCitizenIdWithAi</code>)
Decision tree (<code>lib/core/models/incident_risk.dart</code>)	Risk score ng incident na nakikita ng staff. Hindi ito AI model. Rule-based ito na isinulat sa code, at hindi nito binabago ang <code>priority</code>

Advisory lang ang resulta ng AI. Admin pa rin ang nag-a-approve o nagre-reject ng bawat citizen.

7.5 Backend (Firebase)

Technology	Para saan
Firebase Authentication	Accounts at login: email/password, Google Sign-In, at Facebook Login
Cloud Firestore	Database ng <code>users</code> , <code>incidents</code> , <code>notifications</code> , <code>activity_logs</code> at <code>presence</code> . Dito rin binabasa ng ESP32 ang mga bagong report
Firestore Security Rules	Access depende sa role, at pag-check ng report (fields, priority, sakop ng Rosario, laki ng litrato)
Cloud Functions (Node.js 20, <code>firebase-functions</code> , <code>firebase-admin</code>)	Server code: pagpapadala ng emergency push notification sa BFP (<code>sendBfpEmergencyPush</code>) at ID review
Firebase Cloud Messaging (FCM)	Push notifications sa phone ng BFP personnel
Firebase Hosting	Dito naka-host ang website (web version ng app)

Hindi ginagamit ang Firebase Storage kasi kailangan nito ng bayad na Blaze plan. Kaya naka-save ang mga litrato mismo sa loob ng Firestore document bilang compressed image.

Ganoon din, hindi puwedeng mag-upload ng `.apk` sa Firebase Hosting sa libreng Spark plan. Kaya ang **"Download Android APK"** button sa login page ay nakaturo sa isang pampublikong file sa **Google Drive**. Kapag may bagong APK, pinapalitan lang ang file sa Drive gamit ang "Manage versions", para hindi magbago ang link.

7.6 Hardware at firmware

Technology	Para saan
Arduino (C++) sa ESP32	Ang firmware ng alarm
WiFiManager	Pagpili ng Wi-Fi gamit ang phone, sa hotspot na <code>BFP-Alarm-Setup</code>
ArduinoJson	Paggawa at pagbasa ng JSON na pinapadala at natatanggap galing Firebase
Firebase REST APIs	Pag-login ng device (Identity Toolkit) at pagbasa ng <code>incidents</code> (Firestore <code>runQuery</code>)
FreeRTOS tasks (kasama na sa ESP32)	Pagpapatakbo ng network task at alarm loop sa magkahiwalay na core
arduino-cli (kasama sa Arduino IDE)	Pag-compile at pag-upload ng firmware mula sa terminal (FQBN <code>esp32:esp32:esp32</code>)

7.7 Mga tool para sa developer

Technology	Para saan
Node.js + <code>firebase-admin</code> (<code>tools/firestore-seed</code>)	Mga script para sa paghahanda ng database: paglalagay ng sample data, paggawa ng Super Admin, at pagbura ng test accounts o incidents
Firebase CLI (<code>firebase-tools</code>)	Pag-deploy ng website, Firestore rules at Cloud Functions
Python <code>markdown</code> + headless Chrome/Edge (<code>tools/docs-pdf/build_pdf.py</code>)	Paggawa ng PDF na ito mula sa Markdown file
Git + GitHub	Pag-save ng bawat bersyon ng code

8. Mga integration (paano nagkakabit-kabit ang lahat)

Ang **integration** ay ang pagkakabit ng app sa ibang system o serbisyo para magtulungan sila. Isipin ang app bilang **bahay**, at ang bawat integration bilang **lina ng kuryente, tubig o internet** na nakakabit dito galing sa labas. Ito ang lahat ng nakakabit sa BFP Rosario GIS:

Integration	Saan nakakabit	Paano nag-uusap	Para saan
Firestore Authentication	App, ESP32	Firestore SDK sa app; REST API (Identity Toolkit) sa ESP32	Login ng lahat ng account
Google Sign-In	App → Firestore Auth	<code>google_sign_in</code> sa phone, tapos ipinapasa ang credential sa Firestore; popup sa web	Mabilis na pag-login gamit ang Gmail
Facebook Login	App → Firestore Auth	<code>flutter_facebook_auth</code> sa phone; popup sa web	Pag-login gamit ang Facebook
Cloud Firestore	App, ESP32, Cloud Functions	SDK sa app; REST <code>runQuery</code> sa ESP32; Admin SDK sa Functions	Ang gitnang database ("bulletin board")
Cloud Functions + FCM	Firestore → phone ng BFP	Tumatakbo ang function kapag may bagong <code>notifications</code> document, tapos nagpapadala ng push	Emergency push notification
Google Gemini (Firestore AI Logic)	App	<code>firebase_ai</code> , direkta mula sa app (walang sariling server)	Review ng ID at severity ng litrato
OpenStreetMap tiles	Mapa ng app	<code>flutter_map</code> na kumukuha ng tiles sa internet	Itsura ng mapa
Photon	Report form	HTTPS sa <code>photon.komoot.io</code>	Type-ahead na paghahanap ng lugar
Nominatim	Report form	HTTPS sa <code>nominatim.openstreetmap.org</code>	Address mula sa coordinates; backup ng search
Device GPS	App	<code>geolocator</code> (phone o browser)	Kasalukuyang lokasyon ng citizen
Camera	App	<code>camera</code> sa phone; <code><video></code> ng browser sa web	ID auto-scanner at litrato ng incident
Firestore Hosting	Website	Firestore CLI deploy	Web version ng app
Google Drive	Login page	Pampublikong download link	Pag-download ng Android APK
ESP32 alarm	Firestore	Wi-Fi + HTTPS, tanong kada 3 segundo	Siren sa fire station
Telepono	App	<code>url_launcher</code> (<code>tel:</code> link)	Pagtawag sa hotline ng station

Mga bagong integration sa bersyong ito:

- **Photon place search:** type-ahead na paghahanap ng lugar sa report form, na may Tagalog aliases.
- **Center-pin location picker:** sa mapa ng report form, ang mapa ang dina-drag at nakapirmi ang pin sa gitna.
- **Presence (`presence/{uid}`):** live na bilang ng BFP personnel na online, para sa "Personnel on duty".
- **ID auto-scanner:** gamit ang camera ng phone at ng browser, kasama ang mabilis na pre-check ni Gemini.

- **Verification queue na may ID viewer:** nakikita ng admin ang mismong ID at ang AI review bago mag-approve.
- **Google Drive para sa APK:** kapalit ng Firebase Hosting, na hindi tumatanggap ng .apk sa Spark plan.
- **ESP32 alarm:** direktang nagbabasa sa Firestore gamit ang REST API at sariling device account.

9. Wrap-up: buod ng lahat ng ginamit

#	Technology	Grupo	Ano ito	Ginamit para sa
1	Google Gemini AI (Firebase AI Logic)	AI	AI model ng Google na umiintindi ng litrato	Pag-check ng government ID at severity ng litrato ng incident
2	Nominatim	Mapa / Location	Geocoding service ng OpenStreetMap	Address at barangay mula sa coordinates; backup ng search
3	OpenStreetMap	Mapa / Location	Libreng mapa ng mundo ("Wikipedia ng mapa")	Itura ng mapa: kalsada at mga lugar
4	Photon	Mapa / Location	Type-ahead search ng mga lugar	Paghahanap ng lugar sa report form, may Tagalog aliases
5	flutter_map	Mapa / Location	Flutter package para sa mapa	Mapa ng incidents, fire station at center pin
6	Flutter (Dart)	App	Cross-platform na app framework	Isang code para sa Android, iOS, web at desktop
7	Firebase Authentication	Backend	Accounts at login	Login ng citizen, barangay, BFP, admin at ESP32
8	Google Sign-In	Login	Login gamit ang Gmail	Madaling pag-login ng citizen
9	Facebook Login	Login	Login gamit ang Facebook	Isa pang madaling pag-login
10	Cloud Firestore	Backend	Real-time na NoSQL (document) database	users, incidents, notifications, activity_logs, presence
11	Firestore Security Rules	Backend	Patakaran ng database ("guwardiya")	Access ayon sa role at pag-check ng bawat report
12	Cloud Functions (Node.js 20)	Backend	Code na tumatakbo sa server	Emergency push sa BFP at backup na ID review
13	Firebase Cloud Messaging	Backend	Push notification service	Abiso sa phone ng BFP kapag verified ang report
14	Firebase Hosting	Backend	Web hosting	Website ng app (gis-cross-platform.web.app)
15	Google Drive	Distribution	Online storage	Download link ng Android APK
16	Geolocator (GPS)	Phone feature	GPS ng phone o browser	Kasalukuyang lokasyon ng nagre-report
17	Camera at ID auto-scanner	Phone feature	Camera + sariling scanner ng app	Kusang pagkuha ng ID at litrato ng incident
18	Decision tree	Logic	Rule-based na scoring (hindi AI)	Risk score (Low, Medium, High) para sa staff
19	ESP32	Hardware	Microcontroller na may Wi-Fi	"Utak" ng alarm; tanong sa Firestore kada 3 segundo
20	Arduino (C++)	Firmware	Programming language ng ESP32	Firmware ng incident alarm
21	WiFiManager	Firmware	Wi-Fi setup library	Pagpalit ng Wi-Fi gamit ang hotspot na BFP-Alarm-Setup
22	ArduinoJson	Firmware	JSON library	Pagbasa ng data galing Firebase

#	Technology	Grupo	Ano ito	Ginamit para sa
23	Firebase REST API	Integration	Pag-access sa Firebase gamit ang HTTPS	Login at pagbasa ng incidents ng ESP32

Sa madaling salita: Nagre-report ang citizen gamit ang **Flutter app**, na may **GPS**, mapa (**OpenStreetMap**, **Photon**, **Nominatim**) at **AI (Gemini)**. Naiipon ang report sa **Firebase**, na nagbabantay ng access at nagpapadala ng abiso sa BFP. Sabay nito, nababasa ng **ESP32** ang report at pinapatunog ang **siren** sa fire station.